

Koku.

Parfum Kesif & Sosyal Platform

Web Programlama - 2. Proje Gereksinimleri

1. Veritabanı (EF Core + DbContext + Migrations + CRUD)
2. Authentication & Authorization (Cookie + Rol Sistemi)
3. UI & Layout (_Layout, ViewData, ViewModel)
4. Validations (Data Annotations)

Bilgisayar Muhendisligi - 2025-2026 Bahar Donemi

01

Veritabani

EF Core Code-First | DbContext | Migrations | CRUD

1. Veritabani

1.1 EF Core Code-First & DbContext

Proje EF Core 9 kullanarak Code-First yaklasimla gelistirilmistir. AppDbContext sinifi tum veritabani tablolarini ve iliskilerini tanimlar.

```
// Data/AppDbContext.cs
public class AppDbContext(DbContextOptions<AppDbContext> options) : DbContext(options)
{
    public DbSet<User> Users { get; set; }
    public DbSet<Perfume> Perfumes { get; set; }
    public DbSet<Collection> Collections { get; set; }
    public DbSet<Review> Reviews { get; set; }
    public DbSet<Like> Likes { get; set; }
    public DbSet<Follow> Follows { get; set; }
    public DbSet<Community> Communities { get; set; }
    public DbSet<CommunityMember> CommunityMembers { get; set; }
    public DbSet<Message> Messages { get; set; }
    public DbSet<Notification> Notifications { get; set; }
    // ... 12 tablo toplam
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Unique constraint ornegi:
        modelBuilder.Entity<Review>()
            .HasIndex(r => new { r.UserId, r.PerfumeId }).IsUnique();
        // Cascade delete ornegi:
        modelBuilder.Entity<Notification>()
            .HasOne(n => n.Recipient).WithMany(u => u.ReceivedNotifications)
            .HasForeignKey(n => n.RecipientId).OnDelete(DeleteBehavior.Cascade);
    }
}
```

1.2 Migrations (3 Asama)

Migration Adi	Tarih	Icerik
InitialCreate	08.05.2026	Tum tablolar sifirdan olusturuldu
CollectionMultiStatus	25.05.2026	Owned+Wishlist+Tried birden isaretleme
AddPerfumeSearchIndex	25.05.2026	pg_trgm, GENERATED kolon, GIN index

Migration ornegi:

```
// Migrations/20260508195003_InitialCreate.cs (bir kesit)
migrationBuilder.CreateTable(
    name: "Users",
    columns: table => new {
        Id = table.Column<int>(nullable: false)
            .Annotation("Npgsql:ValueGenerationStrategy", ...),
        Username = table.Column<string>(maxLength: 50, nullable: false),
        Email = table.Column<string>(maxLength: 255, nullable: false),
        PasswordHash = table.Column<string>(nullable: false),
    },
);
```

```
constraints: table => {  
    table.PrimaryKey("PK_Users", x => x.Id);  
}  
);
```

1.3 CRUD Islemleri

Tum temel CRUD (Create, Read, Update, Delete) islemleri EF Core üzerinden uygulanmistir. Asagida farkli controller'lardan somut ornekler verilmistir.

CREATE - Yorum Olusturma (ReviewsController.cs)

```
[HttpPost]
public async Task<IActionResult> Create(int perfumeId, int rating, string body)
{
    db.Reviews.Add(new Review {
        UserId      = currentUser.UserId!.Value,
        PerfumeId   = perfumeId,
        Rating      = rating,
        Body        = body,
        CreatedAt   = DateTime.UtcNow
    });
    await db.SaveChangesAsync();
    return RedirectToAction("Details", "Perfumes", new { id = perfumeId });
}
```

READ - Parfum Listesi (PerfumesController.cs)

```
public async Task<IActionResult> Index(PerfumeIndexViewModel filter)
{
    var query = db.Perfumes.AsQueryable().ApplySearch(filter.Search);
    if (!string.IsNullOrEmpty(filter.Brand))
        query = query.Where(p => p.Brand == filter.Brand);
    filter.Perfumes = await query
        .Skip((filter.Page - 1) * filter.PageSize)
        .Take(filter.PageSize)
        .ToListAsync();
    return View(filter);
}
```

DELETE - Yorum Silme (ReviewsController.cs)

```
[HttpPost]
public async Task<IActionResult> Delete(int id)
{
    var review = await db.Reviews.FindAsync(id);
    if (review == null) return NotFound();
    if (review.UserId != currentUser.UserId) return Forbid();
    db.Reviews.Remove(review);
    await db.SaveChangesAsync();
    return RedirectToAction(...);
}
```

UPDATE - Koleksiyon Statusu Guncelleme (CollectionController.cs)

```
[HttpPost]
public async Task<IActionResult> Toggle(int perfumeId, string status)
{
    var existing = await db.Collections
```

```
        .FirstOrDefaultAsync(c => c.UserId == userId && c.PerfumeId == perfumeId
                                && c.Status == status);
    if (existing != null) db.Collections.Remove(existing);    // varsa sil
    else                db.Collections.Add(new Collection { // yoksa ekle
        UserId = userId, PerfumeId = perfumeId, Status = status
    });
    await db.SaveChangesAsync();
    return ...
}
```

02

Authentication &

Authorization | Cookie Auth | RoI Sistemi

2. Authentication & Authorization

2.1 Cookie Tabanlı Authentication

ASP.NET Core'un Cookie Authentication sistemi kullanılmaktadır. Kullanıcı giriş yaptığında kimlik bilgileri Claims olarak şifrelenir ve tarayıcıya 7 günlük cookie olarak yazılır.

Program.cs:

```
// Program.cs - Servis kaydı
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options => {
        options.LoginPath      = "/Auth/Login";
        options.LogoutPath     = "/Auth/Logout";
        options.ExpireTimeSpan = TimeSpan.FromDays(7);
        options.SlidingExpiration = true; // her istekte sure sıfırlanır
    });
```

AuthController.cs:

```
// AuthController.cs - Giriş akışı
[HttpPost]
public async Task<IActionResult> Login(LoginViewModel model)
{
    if (!ModelState.IsValid) return View(model);

    var user = await db.Users.FirstOrDefaultAsync(u => u.Email == model.Email);
    if (user == null || !BCrypt.Net.BCrypt.Verify(model.Password, user.PasswordHash))
    {
        ModelState.AddModelError("", "E-posta veya şifre hatalı.");
        return View(model);
    }
    await SignInUser(user); // cookie yazılır
    return RedirectToAction("Index", "Feed");
}

private async Task SignInUser(User user)
{
    var claims = new List<Claim> {
        new(ClaimTypes.NameIdentifier, user.Id.ToString()),
        new(ClaimTypes.Name, user.Username),
        new(ClaimTypes.Email, user.Email)
    };
    var identity = new ClaimsIdentity(claims, CookieAuthenticationDefaults.AuthenticationScheme);
    var principal = new ClaimsPrincipal(identity);
    await HttpContext.SignInAsync(CookieAuthenticationDefaults.AuthenticationScheme, principal);
}
```

Kayıt + BCrypt şifre hash:

```
// AuthController.cs - Kayıt
[HttpPost]
public async Task<IActionResult> Register(RegisterViewModel model)
{
    if (!ModelState.IsValid) return View(model);
```



```
var user = new User {
    Username    = model.Username,
    Email       = model.Email,
    PasswordHash = BCrypt.Net.BCrypt.HashPassword(model.Password) // bcrypt
};
db.Users.Add(user);
await db.SaveChangesAsync();
await SignInUser(user);
return RedirectToAction("Index", "Feed");
}
```

2.2 [Authorize] ile Sayfa Koruma

[Authorize] attribute'u ile login olmayan kullanicilarin korunan sayfalara erisimi engellenir; otomatik olarak login sayfasina yonlendirilir.

```
// Ornek: Tum controller korunuyor
[Authorize]
public class FeedController : Controller { ... }

[Authorize]
public class CollectionController : Controller { ... }

[Authorize]
public class CommunitiesController : Controller { ... }

// Ornek: Sadece belirli action'lar korunuyor
public class PerfumesController : Controller
{
    public IActionResult Index() { ... } // herkese acik
    public IActionResult Details() { ... } // herkese acik

    [Authorize] // sadece login kullanıcı parfüm ekleyebilir
    public IActionResult Create() { ... }
}
```

2.3 Rol Tabanlı Yetkilendirme: Admin / Member

Topluluk (Community) sistemi iki farklı kullanıcı rolünü içerir: Admin (topluluğu kuran) ve Member (katılan üye). Her iki rol farklı yetkilere sahiptir.

Model: CommunityMember.cs

```
public class CommunityMember
{
    public int CommunityId { get; set; }
    public int UserId { get; set; }

    [MaxLength(20)]
    public string Role { get; set; } = "member"; // "admin" veya "member"

    public DateTime JoinedAt { get; set; } = DateTime.UtcNow;
}
```

Rol Atama: CommunitiesController.cs

```
// Topluluk kurucusuna otomatik admin atanır:
db.CommunityMembers.Add(new CommunityMember {
    CommunityId = community.Id,
    UserId = userId,
    Role = "admin", // kurucu = admin
});
```

```
// Katılan kullanıcıya member atanır:
db.CommunityMembers.Add(new CommunityMember {
    CommunityId = id,
    UserId      = userId,
    Role        = "member", // katılan = member
});
```

2.4 Admin / Member Yetki Farklari

Yetki Matrisi

Islem	Admin	Member
Gonderi paylasma	Evet	Evet
Kendi gonderisini silme	Evet	Evet
Baskasinin gonderisini silme	Evet	Hayir
Topluluktan ayrilma	Hayir	Evet
Admin badge gosterimi	Evet	Hayir

Controller'da Yetki Kontrolu (CommunitiesController.cs)

```
// Gonderi silme - admin herkesinkini, member sadece kendininkini silebilir
[HttpPost]
public async Task<IActionResult> DeletePost(int communityId, int postId)
{
    var post = await db.CommunityPosts
        .Include(p => p.Community)
        .FirstOrDefaultAsync(p => p.Id == postId);

    // Ne gonderi sahibisin ne de community admin'isin -> 403
    if (post.UserId != userId && post.Community.OwnerId != userId)
        return Forbid();

    db.CommunityPosts.Remove(post);
    await db.SaveChangesAsync();
    return RedirectToAction(nameof(Details), new { id = communityId });
}

// Gonderi paylasma - sadece uye olanlara izin
var isMember = await db.CommunityMembers
    .AnyAsync(m => m.CommunityId == id && m.UserId == userId);
if (!isMember) return Forbid();

// Owner topluluktan ayrilamaz
if (community.OwnerId == userId) {
    TempData["Error"] = "Owner topluluktan ayrilamaz.";
    return RedirectToAction(nameof(Details), new { id });
}
```

View'da Rol Bazli UI Farki (Communities/Details.cshtml)

```
<!-- Admin badge - sadece admin uyelere gosterilir -->
@if (m.Role == "admin")
{
    <span class="badge bg-amber">admin</span>
}

<!-- Sil butonu - sadece gonderi sahibi veya admin gorur -->
```

```
@if (p.UserId == currentUserId || Model.IsOwner)
{
    <form asp-action="DeletePost" method="post">
        <button type="submit">Sil</button>
    </form>
}

<!-- Ayril butonu - sadece admin olmayanlara gosterilir -->
@if (!Model.IsOwner)
{
    <button>@(Model.IsMember  "Ayril" : "Katil")</button>
}
```

03

UI & Layout

`_Layout.cshtml` | `ViewData` | `ViewModel`

3. UI & Layout

3.1 _Layout.cshtml - Ortak Iskelet

Tüm sayfalar Views/Shared/_Layout.cshtml üzerinden ortak bir Header/Sidebar/Footer yapısını paylaşmaktadır. _ViewStart.cshtml sayesinde her view otomatik bu layout'a sarılır.

```
<!-- Views/Shared/_Layout.cshtml (kesit) -->
<!DOCTYPE html>
<html>
<head>
    <title>@ViewData["Title"] - Koku.</title>
    <link rel="stylesheet" href="~/css/site.css" />
    <!-- Bootstrap 5, Bootstrap Icons -->
</head>
<body>
    <!-- SOL SIDEBAR -->
    <nav class="sidebar">
        <a class="navbar-brand">Koku.</a>
        <a asp-controller="Feed" asp-action="Index"
            class="nav-link @(ViewData["ActivePage"] as string == "Feed" "active" : "")">
            Ana Akis
        </a>
        <!-- diger menu ogeleri... -->
        @if (User.Identity.IsAuthenticated == true) {
            <span>@User.Identity.Name</span>
            <form asp-action="Logout"><button>Cikis</button></form>
        }
    </nav>

    <!-- SAYFA ICERIGI -->
    <main>
        @RenderBody() <!-- her sayfanin icerigi buraya gelir -->
    </main>

    <!-- SAYFA OZELINE JS -->
    @await RenderSectionAsync("Scripts", required: false)
</body>
</html>

// Views/_ViewStart.cshtml - her view icin varsayilan layout
@{
    Layout = "_Layout"; // tum sayfalar _Layout.cshtml kullanir
}
```

3.2 ViewData Kullanimi

Controller'dan View'a ViewData üzerinden veri aktarilmektedir. En yaygın kullanım: sayfa başlığı (Title) ve aktif menu vurgusu (ActivePage).

Controller'da ViewData Yazma

```
// FeedController.cs
public async Task<IActionResult> Index()
{
    ViewData["ActivePage"] = "Feed";    // sidebar'da Feed aktif görüntülenir
    var vm = new FeedViewModel { ... };
    return View(vm);
}

// PerfumesController.cs
public async Task<IActionResult> Details(int id)
{
    ViewData["ActivePage"] = "Discover";
    // ...
}
```

View'da ViewData Okuma

```
<!-- Views/Feed/Index.cshtml -->
@{
    ViewData["Title"] = "Ana Akis";        // tarayıcı sekmesi başlığı
    ViewData["ActivePage"] = "Feed";
}

<!-- _Layout.cshtml'de aktif menu vurgusu -->
<a class="nav-link @(ViewData["ActivePage"] as string == "Feed" ? "active" : "")">
    Ana Akis
</a>

<!-- Okunmamış bildirim sayacı (UnreadCountFilter'dan geliyor) -->
@if ((int)(ViewData["UnreadCount"] ?? 0) > 0) {
    <span class="badge">@ViewData["UnreadCount"]</span>
}
```


3.3 ViewModel Kullanimi

Controller'dan View'a veri aktarimi icin tip guvenligi saglayan ViewModel siniflar kullanilmaktadir. Projede 12 ayri ViewModel sinifi mevcuttur.

Ornek 1: PerfumeIndexViewModel.cs

```
// ViewModels/PerfumeIndexViewModel.cs
public class PerfumeIndexViewModel
{
    // Filtreler (formdan gelir)
    public string Search { get; set; }
    public string Brand { get; set; }
    public string Gender { get; set; }
    public string Accord { get; set; }
    public string Sort { get; set; } = "popular";

    // Sayfalama
    public int Page { get; set; } = 1;
    public int PageSize { get; set; } = 24;
    public int TotalCount { get; set; }
    public int TotalPages => (int)Math.Ceiling((double)TotalCount / PageSize);

    // Sonuclar ve filtre secenekleri
    public List<Perfume> Perfumes { get; set; } = [];
    public List<FacetItem> Brands { get; set; } = [];
    public List<FacetItem> Accords { get; set; } = [];
}
```

Ornek 2: CommunityDetailViewModel.cs

```
// ViewModels/CommunityViewModels.cs
public class CommunityDetailViewModel
{
    public Community Community { get; set; } = null!;
    public List<CommunityMember> Members { get; set; } = [];
    public List<CommunityPost> Posts { get; set; } = [];
    public int MemberCount { get; set; }
    public bool IsMember { get; set; } // aktif kullanıcı uye mi
    public bool IsOwner { get; set; } // aktif kullanıcı kurucu mu
}

// Controller'da doldurulur:
var vm = new CommunityDetailViewModel {
    Community = community,
    Members = members,
    MemberCount = await db.CommunityMembers.CountAsync(m => m.CommunityId == id),
    IsMember = members.Any(m => m.UserId == userId),
    IsOwner = community.OwnerId == userId,
};
return View(vm);
```

View'da @model ile ViewModel Baglama

```
<!-- Views/Communities/Details.cshtml -->
@model WebParfum.ViewModels.CommunityDetailViewModel

<h1>@Model.Community.Name</h1>
<p>@Model.MemberCount uye</p>

@if (Model.IsOwner) { <span>(sen)</span> }
@if (Model.IsMember) { <button>Ayril</button> }
else { <button>Katil</button> }
```

04

Validations

Data Annotations | ModelState | Client + Server

4. Validations - Data Annotations

Form validasyonlari Data Annotations ile uygulanmistir. Hem sunucu tarafinda (ModelState.IsValid) hem de istemci tarafinda (jQuery Validation Unobtrusive) calisir.

Kullanilan Annotation'lar

Annotation	Aciklama
[Required]	Alan bos birakilamaz
[MaxLength(n)]	Maksimum karakter siniri
[MinLength(n)]	Minimum karakter siniri
[EmailAddress]	Gecerli e-posta formati zorunludur
[DataType(Password)]	Sifre alani - maskeli gosterim
[Url]	Gecerli URL formati zorunludur
[Display(Name=...)]	Form etiket adi ozellestirme
[Range(min, max)]	Sayisal aralik kontrolu

RegisterViewModel.cs - Tam Ornek

```
// ViewModels/RegisterViewModel.cs
public class RegisterViewModel
{
    [Required(ErrorMessage = "Kullanici adi zorunludur")]
    [MaxLength(50)]
    public string Username { get; set; } = string.Empty;

    [Required(ErrorMessage = "E-posta zorunludur")]
    [EmailAddress]
    public string Email { get; set; } = string.Empty;

    [Required(ErrorMessage = "Sifre zorunludur")]
    [MinLength(6, ErrorMessage = "Sifre en az 6 karakter olmalidir")]
    [DataType(DataType.Password)]
    public string Password { get; set; } = string.Empty;
}
```

LoginViewModel.cs

```
public class LoginViewModel
{
    [Required(ErrorMessage = "E-posta zorunludur")]
    [EmailAddress]
    public string Email { get; set; } = string.Empty;

    [Required(ErrorMessage = "Sifre zorunludur")]
    [DataType(DataType.Password)]
    public string Password { get; set; } = string.Empty;
}
```

4.2 Sunucu Tarafi Kontrol (ModelState.IsValid)

Her POST action'inda ModelState.IsValid kontrolu yapilmaktadir. Validasyon gecemezse form kullaniciya hata mesajlari ile birlikte geri gonderilir.

```
// AuthController.cs - Login POST
[HttpPost]
public async Task<IActionResult> Login(LoginViewModel model)
{
    if (!ModelState.IsValid) return View(model); // hatali ise geri don
    // ... devam et
}

// AuthController.cs - Register POST
[HttpPost]
public async Task<IActionResult> Register(RegisterViewModel model)
{
    if (!ModelState.IsValid) return View(model);

    // Ek is mantigi validasyonlari:
    if (await db.Users.AnyAsync(u => u.Email == model.Email))
        ModelState.AddModelError("Email", "Bu e-posta zaten kullanimda.");

    if (await db.Users.AnyAsync(u => u.Username == model.Username))
        ModelState.AddModelError("Username", "Bu kullanıcı adı alınmış.");

    if (!ModelState.IsValid) return View(model); // ekstra hatalar varsa
    // ... kayıt devam et
}

// CommunitiesController.cs - Create POST
[HttpPost]
public async Task<IActionResult> Create(CommunityCreateViewModel model)
{
    if (!ModelState.IsValid) return View(model);
    // ...
}
```

CommunityCreateViewModel.cs - Diger Annotation Ornekleri

```
public class CommunityCreateViewModel
{
    [Required, MaxLength(100), Display(Name = "Topluluk Adı")]
    public string Name { get; set; } = string.Empty;

    [MaxLength(500), Display(Name = "Açıklama")]
    public string Description { get; set; }

    [Url, Display(Name = "Resim URL")]
    public string ImageUrl { get; set; }
}
```

View'da Hata Gosterimi (Auth/Register.cshtml)

```
@model WebParfum.ViewModels.RegisterViewModel

@if (!ViewData.ModelState.IsValid) {
    <div class="alert alert-danger">
        @foreach (var error in ViewData.ModelState.Values
            .SelectMany(v => v.Errors)) {
            <p>@error.ErrorMessage</p>
        }
    </div>
}

<form asp-action="Register" method="post">
    <input asp-for="Username" class="form-control" />
    <span asp-validation-for="Username" class="text-danger"></span>

    <input asp-for="Email" class="form-control" />
    <span asp-validation-for="Email" class="text-danger"></span>

    <input asp-for="Password" class="form-control" />
    <span asp-validation-for="Password" class="text-danger"></span>

    <button type="submit">Kayit Ol</button>
</form>

@section Scripts { @await Html.PartialAsync("_ValidationScriptsPartial") }
```

Model Sinifinda Data Annotations (User.cs)

```
// Models/User.cs
public class User
{
    public int Id { get; set; }

    [Required, MaxLength(50)]
    public string Username { get; set; } = string.Empty;

    [Required, MaxLength(255)]
    public string Email { get; set; } = string.Empty;

    [Required]
    public string PasswordHash { get; set; } = string.Empty;
}
```

Gereksinim Ozeti

Projenin kurs gereksinimlerini karsilama durumu:

OK	EF Core Code-First	AppDbContext + 12 DbSet<T> + OnModelCreating
OK	DbContext	AppDbContext sinifi - tum DB erisimi buradan
OK	Migrations (3 adet)	InitialCreate, CollectionMultiStatus, AddSearchIndex
OK	CRUD - Create	Review, Collection, Community, Message olusturma
OK	CRUD - Read	Parfum listesi, detay, feed, profil sorgulamalari
OK	CRUD - Update	Koleksiyon status toggle, profil guncelleme
OK	CRUD - Delete	Yorum silme, koleksiyondan cikarma, gonderi silme
OK	Cookie Authentication	AddCookie - 7 gun, sliding expiration, BCrypt
OK	Kullanici Giris/Kayit	AuthController - Login + Register + Logout
OK	Role-based - Admin	Topluluk kurucusu: herkesin gonderisini silebilir
OK	Role-based - Member	Uye: sadece kendi gonderisini silebilir
OK	_Layout.cshtml	Ortak sidebar/nav - tum sayfalar kullanir
OK	ViewData	Title ve ActivePage - her view'da kullaniliyor
OK	ViewModel	12 ayri ViewModel sinifi - tip guvenligi saglar
OK	Data Annotations - Required	RegisterViewModel, LoginViewModel, User modeli
OK	Data Annotations - MaxLength	Username, Email, community Name alanlari
OK	Data Annotations - Email	[EmailAddress] - register ve login formlarinda
OK	Data Annotations - MinLength	Sifre: en az 6 karakter zorunlu
OK	ModelState.IsValid	Tum POST action'larinda kontrol yapiliyor